TECHNICAL STANDARD OPERATING PROCEDURE

PostgreSQL 18 Production I/O Monitoring and Troubleshooting Kit

Snapshot / Delta Analysis, SQL Diagnostics, OS Correlation, and 30-Second Incident Flow

Document Attribute	Value
Technology / Platform	PostgreSQL
Target Version	PostgreSQL 18.x
Document Type	Production Monitoring & Troubleshooting SOP
Document Version	1.0
Prepared Date	2026-08-27
Classification	Internal / Controlled Technical Document

Document Control

Field	Value
Document Title	PostgreSQL 18 Production I/O Monitoring and Troubleshooting Kit
Purpose	Provide a repeatable production workflow to identify PostgreSQL I/O producers, classify the I/O, correlate it to SQL and background activity, and determine whether the bottleneck is memory, storage, checkpoints, temp spill, scans, WAL, vacuum, or writeback.
Scope	PostgreSQL 18 clusters on Linux or comparable production platforms. SQL is portable unless explicitly noted as an OS command.
Technology / Platform	PostgreSQL 18.x
Target Version	18.x; version-specific columns are called out
Document Version	1.0
Primary Documentation Source	PostgreSQL 18 official documentation
Intended Audience	DBAs, SREs, platform engineers, database performance engineers, production support
Document Owner	Database / Platform Engineering
Classification	Internal / Controlled Technical Document
Review Trigger	Major PostgreSQL upgrade, monitoring architecture change, incident postmortem finding, or at least annually

Revision History

Version	Date	Description
1.0	2026-08-27	Initial release

Source and Accuracy Statement

This SOP was prepared and cross-checked against PostgreSQL 18 official documentation for cumulative statistics, pg_stat_io, pg_stat_statements, runtime statistics, checkpointing, vacuuming, and related monitoring features. Production validation is still required against the exact PostgreSQL minor release, operating system, storage platform, infrastructure, configuration, extensions, security requirements, and workload. Metrics are cumulative unless a delta interval is explicitly calculated; interpretation must account for statistics resets and configuration such as track_io_timing.

TABLE OF CONTENTS / INDEX

1. Introduction

PostgreSQL I/O incidents are rarely solved by looking at a single counter. The fastest production method is to identify who generated the I/O, determine what object and context it targeted, correlate the change to SQL or maintenance activity, then confirm whether the pressure is caused by cache misses, temp spill, WAL, checkpoints, vacuum, sequential scans, or the storage layer itself.

The kit uses a snapshot-and-delta model. A scheduled collector stores cumulative PostgreSQL statistics at a fixed interval, and reports compare two snapshots. This avoids misleading conclusions from counters that may have accumulated for days or weeks.

1.1 Objectives

- Find the highest I/O-producing backend classes and SQL statements.
- Separate relation, temporary, and WAL I/O where PostgreSQL exposes the distinction.
- Measure cache effectiveness without treating one global ratio as a universal performance target.
- Detect memory pressure indicators such as buffer evictions and temp spill.
- Detect checkpoint and background-writer pressure.
- Correlate database metrics with operating-system device latency and saturation.
- Provide safe, fast incident commands that do not reset production statistics.

1.2 Important PostgreSQL 18 Version Notes

PostgreSQL 18 expands I/O observability: pg_stat_io reports byte counters such as read_bytes, write_bytes, and extend_bytes; WAL I/O is represented in pg_stat_io; and per-backend I/O can be queried with pg_stat_get_backend_io(). Older monitoring SQL that assumes the PostgreSQL 16/17 pg_stat_io column set must be version-checked before use.

Safety: Do not run pg_stat_reset(), pg_stat_statements_reset(), pg_stat_reset_shared(), CHECKPOINT, VACUUM FULL, REINDEX, or cache-dropping OS commands as part of routine incident collection. These actions alter state, destroy baselines, or can add load.

2. Architecture and Data Flow

```
PostgreSQL cluster
  |
  +--> cumulative statistics (pg_stat_io, pg_stat_database,
  |    pg_stat_bgwriter, pg_stat_checkpointer, pg_stat_wal)
  |
  +--> SQL statistics (pg_stat_statements)
  |
  +--> activity / progress views (pg_stat_activity, progress views)
  |
  v
io_snapshots (every 60 seconds)
  |
  v
```

```
io_delta (difference between two snapshots)
  |
  +--> top I/O producers
  +--> cache / eviction indicators
  +--> temp spill detector
  +--> sequential-scan detector
  +--> checkpoint / writeback analysis
  +--> WAL and autovacuum correlation
  |
  v
OS correlation (iostat / vmstat / pidstat / filesystem)
  |
  v
Incident diagnosis and corrective action
```

2.1 Data Interpretation Rules

- Treat PostgreSQL statistics as cumulative counters and calculate rates/deltas over a known interval.
- Check stats_reset timestamps before comparing samples. A reset invalidates a delta across that boundary.
- I/O timing columns remain zero unless the relevant timing setting is enabled.
- pg_stat_io is cluster-wide by backend type/context/object; pg_stat_statements is the primary SQL-level correlation source.
- A high read count does not automatically mean slow storage. Confirm latency with timing data and OS metrics.
- A high cache-hit ratio does not prove healthy performance; a small working set can still suffer latency, temp spill, WAL pressure, or checkpoint stalls.

3. Prerequisites and Safe Enablement

3.1 Pre-Implementation Checklist

- [ ] Confirm PostgreSQL major/minor version with SELECT version().
- [ ] Confirm pg_stat_statements is approved and installed in the target database.
- [ ] Confirm shared_preload_libraries contains pg_stat_statements before planning a restart.
- [ ] Confirm compute_query_id is auto/on or otherwise supplied.
- [ ] Measure timing overhead with pg_test_timing before enabling I/O timing on latency-sensitive systems.
- [ ] Confirm monitoring role permissions follow least privilege.
- [ ] Confirm the snapshot repository has retention and cleanup rules.
- [ ] Confirm the monitoring database/table is excluded from unnecessary external replication/backup if policy allows.
- [ ] Confirm OS tools such as iostat, vmstat, pidstat, df, and lsblk are available to operators.

3.2 Recommended Monitoring Settings

```
-- Review current settings first.
SHOW shared_preload_libraries;
SHOW compute_query_id;
```

```
SHOW track_io_timing;
SHOW track_wal_io_timing;
SHOW track_counts;

-- Typical production choices, subject to local performance testing:
-- shared_preload_libraries = 'pg_stat_statements'
-- compute_query_id = 'auto'
-- track_io_timing = on
-- track_wal_io_timing = on

-- pg_stat_statements must be created in each database where its SQL view is used.
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

Change Control: shared_preload_libraries is a server-start setting. Adding pg_stat_statements requires a PostgreSQL restart. Enabling timing has measurable clock-reading overhead on some systems; test before broad rollout.

3.3 Least-Privilege Monitoring Role

```
-- Run as an authorized administrator and adapt to local policy.
CREATE ROLE pg_monitoring LOGIN PASSWORD '<MANAGED_SECRET>';
GRANT pg_monitor TO pg_monitoring;

-- If the snapshot schema is stored in a dedicated database/schema:
GRANT CONNECT ON DATABASE <MONITORING_DB> TO pg_monitoring;
GRANT USAGE ON SCHEMA monitor TO pg_monitoring;
GRANT SELECT, INSERT ON ALL TABLES IN SCHEMA monitor TO pg_monitoring;
ALTER DEFAULT PRIVILEGES IN SCHEMA monitor
  GRANT SELECT, INSERT ON TABLES TO pg_monitoring;
```

Security: Use a secrets manager or platform credential mechanism. Do not place plaintext database passwords in cron files, shell history, source control, or monitoring SQL.

4. Monitoring Kit Implementation

4.1 Snapshot Schema

The repository below stores the key pg_stat_io dimensions and PostgreSQL 18 byte/timing counters. It also stores selected database, checkpointer, background-writer, and WAL counters. Keep the repository small and retention-bound; the goal is operational deltas, not indefinite telemetry storage.

```
CREATE SCHEMA IF NOT EXISTS monitor;

CREATE TABLE IF NOT EXISTS monitor.io_snapshots (
   snapshot_ts      timestamptz NOT NULL DEFAULT clock_timestamp(),
```

```
    backend_type     text NOT NULL,
    object           text NOT NULL,
    context          text NOT NULL,
    reads            bigint,
    read_bytes       numeric,
    read_time_ms     double precision,
    writes           bigint,
    write_bytes      numeric,
    write_time_ms    double precision,
    writebacks       bigint,
    writeback_time_ms double precision,
    extends          bigint,
    extend_bytes     numeric,
    extend_time_ms   double precision,
    op_bytes         bigint,
    hits             bigint,
    evictions        bigint,
    reuses           bigint,
    fsyncs           bigint,
    fsync_time_ms    double precision,
    stats_reset      timestamptz
);

CREATE INDEX IF NOT EXISTS io_snapshots_ts_idx
    ON monitor.io_snapshots (snapshot_ts);

CREATE TABLE IF NOT EXISTS monitor.db_snapshots AS
SELECT clock_timestamp() AS snapshot_ts, d.*
FROM pg_stat_database d WITH NO DATA;

CREATE TABLE IF NOT EXISTS monitor.checkpointer_snapshots AS
SELECT clock_timestamp() AS snapshot_ts, c.*
FROM pg_stat_checkpointer c WITH NO DATA;

CREATE TABLE IF NOT EXISTS monitor.bgwriter_snapshots AS
SELECT clock_timestamp() AS snapshot_ts, b.*
FROM pg_stat_bgwriter b WITH NO DATA;

CREATE TABLE IF NOT EXISTS monitor.wal_snapshots AS
SELECT clock_timestamp() AS snapshot_ts, w.*
FROM pg_stat_wal w WITH NO DATA;
```

4.2 Version-Adaptive pg_stat_io Snapshot Function

PostgreSQL 18 changed pg_stat_io columns. The function below discovers whether byte columns exist and stores them. The op_bytes target column is retained only as a compatibility field and is populated when an older server exposes it.

```
CREATE OR REPLACE FUNCTION monitor.capture_io_snapshot()
RETURNS void
LANGUAGE plpgsql
SECURITY INVOKER
AS $$
DECLARE
  has_read_bytes boolean;
  has_op_bytes   boolean;
  sql_text      text;
BEGIN
  SELECT EXISTS (
    SELECT 1 FROM pg_attribute
    WHERE attrelid = 'pg_catalog.pg_stat_io'::regclass
      AND attname = 'read_bytes' AND NOT attisdropped
  ) INTO has_read_bytes;

  SELECT EXISTS (
    SELECT 1 FROM pg_attribute
    WHERE attrelid = 'pg_catalog.pg_stat_io'::regclass
      AND attname = 'op_bytes' AND NOT attisdropped
  ) INTO has_op_bytes;

  sql_text := format($q$
    INSERT INTO monitor.io_snapshots
    (snapshot_ts, backend_type, object, context,
     reads, read_bytes, read_time_ms,
     writes, write_bytes, write_time_ms,
     writebacks, writeback_time_ms,
     extends, extend_bytes, extend_time_ms,
     op_bytes, hits, evictions, reuses, fsyncs, fsync_time_ms, stats_reset)
    SELECT clock_timestamp(), backend_type, object, context,
           reads, %s, read_time,
           writes, %s, write_time,
           writebacks, writeback_time,
           extends, %s, extend_time,
           %s, hits, evictions, reuses, fsyncs, fsync_time, stats_reset
    FROM pg_stat_io
  $q$,
    CASE WHEN has_read_bytes THEN 'read_bytes' ELSE 'NULL::numeric' END,
    CASE WHEN has_read_bytes THEN 'write_bytes' ELSE 'NULL::numeric' END,
```

```
    CASE WHEN has_read_bytes THEN 'extend_bytes' ELSE 'NULL::numeric' END,
    CASE WHEN has_op_bytes THEN 'op_bytes' ELSE 'NULL::bigint' END);

  EXECUTE sql_text;

  INSERT INTO monitor.db_snapshots
    SELECT clock_timestamp(), d.* FROM pg_stat_database d;
  INSERT INTO monitor.checkpointer_snapshots
    SELECT clock_timestamp(), c.* FROM pg_stat_checkpointer c;
  INSERT INTO monitor.bgwriter_snapshots
    SELECT clock_timestamp(), b.* FROM pg_stat_bgwriter b;
  INSERT INTO monitor.wal_snapshots
    SELECT clock_timestamp(), w.* FROM pg_stat_wal w;
END;
$$;
```

4.3 One-Minute Collection

```
# Example cron entry. Use .pgpass or a managed credential, not PGPASSWORD.
* * * * * psql -X -v ON_ERROR_STOP=1 -d <MONITORING_DB>   -c "SELECT monitor.capture_io_snapshot();"   >>
/var/log/postgresql/io_snapshot.log 2>&1
```

4.4 Retention Cleanup

```
-- Example: keep 14 days. Tune to your incident-retention policy.
DELETE FROM monitor.io_snapshots
WHERE snapshot_ts < now() - interval '14 days';
DELETE FROM monitor.db_snapshots
WHERE snapshot_ts < now() - interval '14 days';
DELETE FROM monitor.checkpointer_snapshots
WHERE snapshot_ts < now() - interval '14 days';
DELETE FROM monitor.bgwriter_snapshots
WHERE snapshot_ts < now() - interval '14 days';
DELETE FROM monitor.wal_snapshots
WHERE snapshot_ts < now() - interval '14 days';
```

5. 30-Second Production Troubleshooting Flow

When an I/O alert fires, use the following order. The sequence starts with attribution, then narrows the cause, then confirms whether PostgreSQL or the storage platform is the limiting layer.

Step	Question	Use
1	Who is generating the most I/O?	Report 1
2	What are they accessing: relations, WAL, or temporary data?	Report 2

Step	Question	Use
3	Why are they doing it: normal workload, bulk operation, vacuum, checkpoint, or maintenance?	Report 3
4	Is the workload being served from memory or requiring reads?	Report 4
5	Are there memory-pressure indicators such as evictions or temp spill?	Report 5
6	Is the storage layer itself slow or saturated?	Report 6
7	Are sequential scans contributing significant read volume?	Report 7
8	Are queries creating temporary files?	Report 8
9	Are checkpoints or backend writes causing spikes?	Report 9
10	Is autovacuum generating significant I/O?	Report 10
11	Is WAL generation, archiving, or replication retention growing unexpectedly?	Report 11
12	Is background writeback ineffective or frequently hitting its limit?	Report 12
13	Are long-running transactions preventing cleanup and amplifying I/O?	Report 13
14	Are table/index bloat or stale statistics increasing work?	Report 14
15	Is the filesystem nearly full, inode-starved, or showing device errors?	Report 15

6. Core Diagnostic Reports

6.1 Report 1 - Top I/O Producers by Backend Type

```
WITH bounds AS (
  SELECT max(snapshot_ts) AS t2,
         max(snapshot_ts) - interval '60 seconds' AS target_t1
  FROM monitor.io_snapshots
), times AS (
  SELECT b.t2,
         (SELECT max(snapshot_ts) FROM monitor.io_snapshots s
          WHERE s.snapshot_ts <= b.target_t1) AS t1
  FROM bounds b
), a AS (
  SELECT backend_type, object, context,
         sum(coalesce(read_bytes, reads * op_bytes, 0)) AS read_b,
         sum(coalesce(write_bytes, writes * op_bytes, 0)) AS write_b,
         sum(coalesce(extend_bytes, extends * op_bytes, 0)) AS extend_b
```

```
  FROM monitor.io_snapshots, times
  WHERE snapshot_ts = times.t1
  GROUP BY 1,2,3
), b AS (
  SELECT backend_type, object, context,
        sum(coalesce(read_bytes, reads * op_bytes, 0)) AS read_b,
        sum(coalesce(write_bytes, writes * op_bytes, 0)) AS write_b,
        sum(coalesce(extend_bytes, extends * op_bytes, 0)) AS extend_b
  FROM monitor.io_snapshots, times
  WHERE snapshot_ts = times.t2
  GROUP BY 1,2,3
)
SELECT b.backend_type, b.object, b.context,
      pg_size_pretty(greatest(b.read_b-a.read_b,0)::bigint) AS read_delta,
      pg_size_pretty(greatest(b.write_b-a.write_b,0)::bigint) AS write_delta,
      pg_size_pretty(greatest(b.extend_b-a.extend_b,0)::bigint) AS extend_delta
FROM b JOIN a USING (backend_type, object, context)
ORDER BY greatest(b.read_b-a.read_b,0) + greatest(b.write_b-a.write_b,0) DESC
LIMIT 20;
```

Interpretation: client backend reads/writes point toward foreground SQL; autovacuum worker activity points toward maintenance; checkpointer/background writer activity points toward dirty-buffer flushing; WAL rows indicate log I/O. A single interval is evidence, not a trend - compare several intervals around the alert.

6.2 Report 2 - What Object/Context Is Driving I/O?

```
SELECT backend_type, object, context,
      reads, read_bytes, read_time,
      writes, write_bytes, write_time,
      writebacks, extends, extend_bytes,
      hits, evictions, reuses, fsyncs, fsync_time
FROM pg_stat_io
ORDER BY coalesce(read_bytes,0) + coalesce(write_bytes,0) DESC NULLS LAST;
```

Use object and context together. Context describes the I/O access pattern such as normal, bulkread, bulkwrite, or vacuum where applicable. PostgreSQL 18 also exposes WAL I/O in this view.

6.3 Report 3 - Why Is It Happening Right Now?

```
SELECT pid, usename, datname, backend_type, state,
      wait_event_type, wait_event,
      now() - query_start AS query_age,
      now() - xact_start AS xact_age,
      left(query, 500) AS query
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
  AND state IS DISTINCT FROM 'idle'
ORDER BY query_start NULLS LAST;
```

Correlate active sessions with pg_stat_io backend types and OS device pressure. wait_event_type = IO can be useful, but absence of an I/O wait at one instant does not prove the workload is not I/O-heavy.

6.4 Report 4 - Cache Effectiveness by Database and Table

```
-- Database-level relation block cache ratio.
SELECT datname,
    blks_read,
    blks_hit,
    round(100.0 * blks_hit / nullif(blks_hit + blks_read,0), 2) AS hit_pct
FROM pg_stat_database
WHERE datname IS NOT NULL
ORDER BY blks_read DESC;

-- Table-level heap cache ratio. Use deltas for incident analysis.
SELECT schemaname, relname,
    heap_blks_read, heap_blks_hit,
    round(100.0 * heap_blks_hit /
        nullif(heap_blks_hit + heap_blks_read,0), 2) AS heap_hit_pct
FROM pg_statio_user_tables
ORDER BY heap_blks_read DESC
LIMIT 30;
```

Interpretation: Do not use a fixed cache-hit percentage as a universal pass/fail threshold. Workload size, scan patterns, OS page cache, table size, and query design all matter. Look for changes from the normal baseline and combine with read latency and throughput.

6.5 Report 5 - Memory Pressure Indicators

```
SELECT backend_type, object, context,
    hits, evictions, reuses,
    reads, read_bytes, read_time
FROM pg_stat_io
WHERE coalesce(evictions,0) > 0
  OR coalesce(reuses,0) > 0
ORDER BY evictions DESC NULLS LAST;
```

High or rapidly increasing evictions can indicate shared-buffer pressure, but do not increase shared_buffers blindly. Check working-set size, query access patterns, OS memory, huge pages, connection count, and temp spill first.

6.6 Report 6 - Storage Latency and Saturation

```
-- PostgreSQL timing (requires track_io_timing; WAL timing uses track_wal_io_timing).
SELECT backend_type, object, context,
    reads, read_time,
    CASE WHEN reads > 0 THEN round((read_time / reads)::numeric, 3) END AS avg_read_ms,
```

```
    writes, write_time,
    CASE WHEN writes > 0 THEN round((write_time / writes)::numeric, 3) END AS avg_write_ms,
    fsyncs, fsync_time,
    CASE WHEN fsyncs > 0 THEN round((fsync_time / fsyncs)::numeric, 3) END AS avg_fsync_ms
FROM pg_stat_io
WHERE coalesce(reads,0)+coalesce(writes,0)+coalesce(fsyncs,0) > 0
ORDER BY coalesce(read_time,0)+coalesce(write_time,0)+coalesce(fsync_time,0) DESC;

# Linux correlation - choose the actual PostgreSQL devices.
iostat -xz 1 10
vmstat 1 10
pidstat -d -p $(head -1 "$PGDATA/postmaster.pid") 1 10
lsblk -o NAME,KNAME,TYPE,SIZE,FSTYPE,MOUNTPOINTS
findmnt -T "$PGDATA"
df -hT "$PGDATA"
df -ih $PGDATA
```

Interpretation: confirm latency at both layers. Device await, queue depth, utilization, throughput, filesystem capacity, and PostgreSQL read/write/fsync timing should tell a consistent story. Cloud storage may require provider-specific volume metrics because guest iostat does not always reveal throttling limits.

6.7 Report 7 - Sequential Scan Detector

```
SELECT schemaname, relname,
    seq_scan, seq_tup_read, idx_scan,
    n_live_tup,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_stat_user_tables
WHERE seq_scan > 0
ORDER BY seq_tup_read DESC
LIMIT 30;
```

A sequential scan is not automatically bad. Large analytics, bulk processing, small tables, and low-selectivity predicates can make it correct. Investigate when scan volume, elapsed time, or storage reads rise unexpectedly.

6.8 Report 8 - Temporary File / Spill Detector

```
-- Database-level cumulative temp activity.
SELECT datname, temp_files, pg_size_pretty(temp_bytes) AS temp_written
FROM pg_stat_database
ORDER BY temp_bytes DESC;

-- SQL-level temp writers.
SELECT queryid, calls,
    pg_size_pretty(temp_blks_written * current_setting('block_size')::bigint) AS temp_written,
    round(temp_blk_write_time::numeric,2) AS temp_write_ms,
    left(query, 400) AS query
FROM pg_stat_statements
```

```
WHERE temp_blks_written > 0
ORDER BY temp_blks_written DESC
LIMIT 20;
```

Temp files commonly come from sorts, hashes, materialization, and other executor operations that exceed available memory. Do not solve every spill by globally increasing work_mem; that setting can be consumed by multiple operations in many concurrent sessions.

6.9 Report 9 - Checkpoint Bottleneck / Write Spikes

```
SELECT * FROM pg_stat_checkpointer;
SELECT * FROM pg_stat_bgwriter;

-- Key I/O attribution around checkpoint pressure.
SELECT backend_type, object, context,
    writes, write_bytes, write_time,
    writebacks, writeback_time,
    fsyncs, fsync_time
FROM pg_stat_io
WHERE backend_type IN ('checkpointer','background writer','client backend')
ORDER BY coalesce(write_bytes,0) DESC NULLS LAST;
```

Look for frequent requested checkpoints, high checkpoint write/sync time, client-backend writes, or client-backend fsyncs. PostgreSQL documentation notes that large client-backend write/fsync counts can indicate shared-buffer or checkpointer misconfiguration. Use deltas and correlate with WAL generation and storage latency.

6.10 Report 10 - Autovacuum I/O

```
SELECT pid, datname, relid::regclass AS relation,
    phase, heap_blks_total, heap_blks_scanned, heap_blks_vacuumed,
    index_vacuum_count,
    now() - a.query_start AS runtime
FROM pg_stat_progress_vacuum v
JOIN pg_stat_activity a USING (pid)
ORDER BY runtime DESC;

SELECT backend_type, object, context,
    reads, read_bytes, read_time,
    writes, write_bytes, write_time
FROM pg_stat_io
WHERE backend_type = 'autovacuum worker';
```

If autovacuum is the producer, determine whether the activity is expected cleanup, aggressive/failsafe vacuuming, or a response to a write-heavy table. Check dead tuples, freeze age, table-specific autovacuum settings, and whether long transactions are preventing cleanup.

6.11 Report 11 - WAL Growth, Archiving, and Replication Retention

```
SELECT * FROM pg_stat_wal;
SELECT * FROM pg_stat_archiver;

SELECT slot_name, slot_type, active,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
FROM pg_replication_slots
WHERE restart_lsn IS NOT NULL
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;

SELECT application_name, client_addr, state, sync_state,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_lag_bytes
FROM pg_stat_replication
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) DESC NULLS LAST;
```

Unexpected WAL growth can be caused by write bursts, bulk loads, index creation, full-page images after checkpoints, replication lag, inactive slots, or archive failures. Confirm pg_wal filesystem capacity before it becomes an outage.

6.12 Report 12 - Background Writer Effectiveness

```
SELECT buffers_clean, maxwritten_clean, buffers_alloc, stats_reset
FROM pg_stat_bgwriter;

SELECT backend_type, context,
    writes, write_bytes, write_time,
    writebacks, writeback_time
FROM pg_stat_io
WHERE backend_type IN ('background writer','checkpointer','client backend')
ORDER BY backend_type, context;
```

A rising maxwritten_clean means the background writer repeatedly reached its configured cleaning limit. Evaluate together with client-backend writes, checkpoint behavior, shared-buffer churn, and storage capacity.

7. Additional Production Checklists and Scripts

7.1 Report 13 - Long Transactions and Old Snapshots

```
SELECT pid, usename, datname, state,
    now() - xact_start AS xact_age,
    now() - query_start AS query_age,
    wait_event_type, wait_event,
    left(query,400) AS query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;
```

Long transactions can retain dead tuples and old row versions, increase vacuum work, and contribute to table/index growth. Investigate application behavior before terminating sessions.

7.2 Report 14 - Dead Tuples, Analyze Health, and Candidate Bloat

```
SELECT schemaname, relname,
    n_live_tup, n_dead_tup,
    last_vacuum, last_autovacuum,
    last_analyze, last_autoanalyze,
    vacuum_count, autovacuum_count,
    analyze_count, autoanalyze_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 30;

-- Approximate relation/index size inventory; size alone is not proof of bloat.
SELECT n.nspname AS schema_name, c.relname,
    pg_size_pretty(pg_relation_size(c.oid)) AS relation_size,
    pg_size_pretty(pg_indexes_size(c.oid)) AS indexes_size,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r','m')
  AND n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY pg_total_relation_size(c.oid) DESC
LIMIT 30;
```

Bloat: Core statistics can identify candidates, not exact bloat percentage. Exact bloat analysis often requires extensions or page-level inspection and should be performed under approved production procedures.

7.3 Report 15 - Filesystem Capacity and Error Checklist

```
df -hT "$PGDATA"
df -ih "$PGDATA"
findmnt -T "$PGDATA"
lsblk -f
journalctl -k --since '-30 min' | egrep -i 'I/O error|blk|nvme|scsi|ext4|xfs|reset|timeout'

# PostgreSQL WAL location may be symlinked; resolve it.
readlink -f "$PGDATA/pg_wal"
df -hT $(readlink -f $PGDATA/pg_wal)
```

Privilege: Kernel logs may require elevated OS privileges. Follow local access controls. Do not run repair commands during diagnosis unless the storage/OS team has approved them.

7.4 Top SQL by Physical Read Time

```
SELECT queryid, calls,
    round(total_exec_time::numeric,2) AS total_exec_ms,
    shared_blks_read,
    round(shared_blk_read_time::numeric,2) AS shared_read_ms,
    shared_blks_written,
    round(shared_blk_write_time::numeric,2) AS shared_write_ms,
    temp_blks_written,
    round(temp_blk_write_time::numeric,2) AS temp_write_ms,
    left(query,500) AS query
FROM pg_stat_statements
ORDER BY shared_blk_read_time DESC
LIMIT 20;
```

This report becomes much more useful when track_io_timing is enabled. Compare calls and total time with the incident window; cumulative pg_stat_statements values can otherwise overemphasize historically busy statements.

7.5 Top SQL by WAL Generation

```
SELECT queryid, calls,
    pg_size_pretty(wal_bytes::bigint) AS wal_generated,
    wal_records, wal_fpi,
    round(total_exec_time::numeric,2) AS total_exec_ms,
    left(query,500) AS query
FROM pg_stat_statements
ORDER BY wal_bytes DESC
LIMIT 20;
```

High WAL generation is expected for some write-heavy operations. Investigate sudden changes, especially when they coincide with pg_wal growth, archive lag, replica lag, or storage throttling.

7.6 Blocking and Lock Amplification Check

```
SELECT a.pid AS blocked_pid,
    pg_blocking_pids(a.pid) AS blocking_pids,
    now() - a.query_start AS blocked_for,
    a.wait_event_type, a.wait_event,
    left(a.query,300) AS blocked_query
FROM pg_stat_activity a
WHERE cardinality(pg_blocking_pids(a.pid)) > 0
ORDER BY a.query_start;
```

Locking is not I/O, but blocked sessions can create backlog and then release a burst of work that looks like an I/O spike. Always check blocking during incident triage.

7.7 Table-Level I/O Hotspots

```
SELECT s.schemaname, s.relname,
      s.heap_blks_read, s.heap_blks_hit,
      s.idx_blks_read, s.idx_blks_hit,
      t.seq_scan, t.seq_tup_read, t.idx_scan,
      pg_size_pretty(pg_total_relation_size(t.relid)) AS total_size
FROM pg_statio_user_tables s
JOIN pg_stat_user_tables t
  ON t.relid = s.relid
ORDER BY (s.heap_blks_read + s.idx_blks_read) DESC
LIMIT 30;
```

7.8 Optional Slow-Plan Capture with auto_explain

auto_explain can log plans for slow statements, but it adds overhead, especially with ANALYZE and per-node timing. Use it only through change control and start conservatively.

```
# Example concept for postgresql.conf; validate in a lower environment first.
# session_preload_libraries = 'auto_explain'
# auto_explain.log_min_duration = '2s'
# auto_explain.log_analyze = on
# auto_explain.log_timing = off
# auto_explain.log_buffers = on
# auto_explain.log_nested_statements = off
```

7.9 Production Incident Collection Shell Script

```
#!/usr/bin/env bash
set -euo pipefail
OUTDIR="${1:-/tmp/pg_io_incident_$(date +%Y%m%d_%H%M%S)}"
mkdir -p "$OUTDIR"

psql -X -v ON_ERROR_STOP=1 -Atqc "select version();" > "$OUTDIR/version.txt"
psql -X -v ON_ERROR_STOP=1 -c "select now(), * from pg_stat_io;" > "$OUTDIR/pg_stat_io.txt"
psql -X -v ON_ERROR_STOP=1 -c "select now(), * from pg_stat_checkpointer;" >
"$OUTDIR/pg_stat_checkpointer.txt"
psql -X -v ON_ERROR_STOP=1 -c "select now(), * from pg_stat_bgwriter;" > "$OUTDIR/pg_stat_bgwriter.txt"
psql -X -v ON_ERROR_STOP=1 -c "select now(), * from pg_stat_wal;" > "$OUTDIR/pg_stat_wal.txt"
psql -X -v ON_ERROR_STOP=1 -c "select now(), * from pg_stat_database;" > "$OUTDIR/pg_stat_database.txt"
psql -X -v ON_ERROR_STOP=1 -c "select
pid,usename,datname,backend_type,state,wait_event_type,wait_event,query_start,xact_start,left(query,1000)
query from pg_stat_activity where pid<>pg_backend_pid();" > "$OUTDIR/pg_stat_activity.txt"

command -v iostat >/dev/null && iostat -xz 1 10 > "$OUTDIR/iostat.txt" || true
command -v vmstat >/dev/null && vmstat 1 10 > "$OUTDIR/vmstat.txt" || true
df -hT > "$OUTDIR/df_hT.txt"
```

```
df -ih > "$OUTDIR/df_ih.txt"
lsblk -f > "$OUTDIR/lsblk.txt" 2>&1 || true

echo Incident_bundle: $OUTDIR
```

Privacy: SQL text can contain literals or sensitive business data. Protect incident bundles, limit access, and redact before sharing outside the authorized operations team.

7.10 Checklist - What Else to Check During I/O Incidents

- [ ] Statistics reset occurred during the comparison interval.
- [ ] Connection surge increased concurrent working sets and memory demand.
- [ ] Bulk load, CREATE INDEX, REINDEX, CLUSTER, VACUUM, ANALYZE, backup, or restore activity is running.
- [ ] Logical/physical replication lag is retaining WAL.
- [ ] Archive command or archive library is failing or slow.
- [ ] A replication slot is inactive and retaining WAL.
- [ ] Long transactions or idle-in-transaction sessions are preventing cleanup.
- [ ] Temp tablespaces are on a slow or full filesystem.
- [ ] Tablespaces map hot objects to a constrained device.
- [ ] Cloud volume IOPS/throughput/burst credits or queue limits are reached.
- [ ] Filesystem is near capacity or inode exhaustion.
- [ ] Kernel/device logs show resets, timeouts, or I/O errors.
- [ ] Recent configuration change altered shared_buffers, checkpoint settings, work_mem, autovacuum, huge pages, or storage mount options.
- [ ] A deployment changed query plans, access patterns, batch size, or concurrency.
- [ ] Statistics are stale and planner estimates are poor.
- [ ] Index growth or table growth changed the working set beyond memory capacity.

8. Validation and Expected Results

8.1 Implementation Validation

- [ ] SELECT monitor.capture_io_snapshot(); completes without error.
- [ ] Two snapshots taken one minute apart show non-negative deltas when no statistics reset occurs.
- [ ] PostgreSQL 18 byte columns populate in monitor.io_snapshots.
- [ ] pg_stat_statements returns rows in the target database.
- [ ] I/O timing fields become non-zero under real I/O when timing is enabled.
- [ ] Retention cleanup removes only data older than the approved window.
- [ ] Monitoring role can read required views and write only to the approved monitoring schema.
- [ ] Cron/systemd collection logs failures and does not overlap excessively.

8.2 Expected Result Patterns

Pattern	Likely Direction	Confirm With

Pattern	Likely Direction	Confirm With
Client backend read bytes rise; read latency rises	Foreground SQL / cache miss / scan pressure	pg_stat_statements, table I/O, iostat
Temp bytes and temp_blks_written rise	Sort/hash spill	pg_stat_statements temp columns, plan
Checkpointer writes and checkpoint time rise	Checkpoint pressure	pg_stat_checkpointer, WAL rate, storage
Client backend fsyncs/writes rise	Writeback/checkpointer configuration concern	pg_stat_io, checkpoint settings
Autovacuum worker I/O rises	Maintenance cleanup	pg_stat_progress_vacuum, dead tuples
WAL I/O and wal_bytes rise	Write burst / full-page images / maintenance	pg_stat_statements WAL, slots, archiver
Evictions rise rapidly	Shared buffer churn / working-set pressure	pg_stat_io, OS memory, query access pattern
DB timing normal but device metrics bad	Storage/platform issue outside PostgreSQL	iostat/cloud volume metrics/kernel logs

9. Alerting and Operational Thresholds

Use baselines and service objectives rather than universal hard-coded thresholds. PostgreSQL workloads differ significantly. Alert on sustained deviations, rate-of-change, and user impact. The examples below are signals to baseline, not vendor-prescribed limits.

Signal	Suggested Alert Logic	Why
Read/write latency	Sustained above workload baseline and application SLO impact	Detect storage or queueing degradation
Temp bytes rate	Large increase vs normal for database/query	Detect memory spills and plan changes
Checkpoint requested rate	Increase above normal or repeated spikes	Detect WAL/checkpoint pressure
Client backend writes/fsyncs	Material increase vs baseline	Detect writeback/checkpointer issues
WAL generation rate	Unexpected sustained increase	Detect write bursts and

Signal	Suggested Alert Logic	Why
		retention risk
Retained WAL by slot	Growing toward pg_wal capacity budget	Prevent disk exhaustion
Filesystem free space	Capacity-based warning/critical levels with growth forecast	Prevent outage
Eviction rate	Sustained increase with read pressure	Detect buffer churn
Autovacuum runtime/backlog	Long-running or falling behind write rate	Prevent bloat/freeze risk

10. Troubleshooting Scenarios

10.1 Sudden Read I/O Spike

Item	Details
Symptom	Read latency and read bytes rise during normal traffic.
What to Check	Check pg_stat_io delta by backend type; top SQL by shared read time; sequential scans; table I/O; iostat.
Expected Result	A small set of SQL/table objects dominates the new read volume.
Possible Root Causes	Plan regression, new batch, cache churn, missing/ineffective index, working-set growth.
Corrective Action	Validate plan and statistics; tune query/index only after confirming; review memory/storage if broad workload-wide pressure.
Validation	Repeat snapshot/delta and confirm application latency, PostgreSQL timing, and OS device metrics return toward baseline.

10.2 Temp Disk Saturation

Item	Details
Symptom	Temp filesystem fills or storage latency spikes while CPU may remain moderate.
What to Check	Check pg_stat_database temp_bytes delta and pg_stat_statements temp_blks_written.
Expected Result	One or more SQL statements show high temp writes.

Item	Details
Possible Root Causes	Sort/hash spill, high concurrency, plan change, insufficient operation memory.
Corrective Action	Tune the query/plan; consider targeted work_mem/session settings rather than a large global increase.
Validation	Repeat snapshot/delta and confirm application latency, PostgreSQL timing, and OS device metrics return toward baseline.

10.3 Checkpoint Write Storm

Item	Details
Symptom	Periodic latency spikes align with heavy writes/fsync.
What to Check	Check pg_stat_checkpointer delta, pg_stat_io checkpointer/client writes, WAL rate, iostat.
Expected Result	Checkpoint activity and storage queue rise together.
Possible Root Causes	Checkpoint frequency, WAL burst, undersized checkpoint smoothing, constrained storage.
Corrective Action	Review checkpoint_timeout, max_wal_size, checkpoint_completion_target and storage capacity under change control.
Validation	Repeat snapshot/delta and confirm application latency, PostgreSQL timing, and OS device metrics return toward baseline.

10.4 Autovacuum Dominates I/O

Item	Details
Symptom	Maintenance workers are top I/O producers.
What to Check	Check progress vacuum, dead tuples, freeze ages, long transactions, table-level autovacuum settings.
Expected Result	Specific tables or aggressive vacuum dominate.
Possible Root Causes	Write-heavy table, long transactions, wraparound prevention, poor table-specific tuning.
Corrective Action	Remove blockers, tune table-specific autovacuum, schedule heavy maintenance where appropriate.

Item	Details
Validation	Repeat snapshot/delta and confirm application latency, PostgreSQL timing, and OS device metrics return toward baseline.

10.5 WAL Directory Growth

Item	Details
Symptom	pg_wal grows unexpectedly.
What to Check	Check replication slots, archiver, replicas, wal_bytes, filesystem capacity.
Expected Result	Inactive slot, archive failures, or replica lag retains WAL.
Possible Root Causes	Consumer outage, archive destination issue, heavy write workload.
Corrective Action	Restore consumer/archive path; manage slots per policy; never delete WAL files manually.
Validation	Repeat snapshot/delta and confirm application latency, PostgreSQL timing, and OS device metrics return toward baseline.

11. Production Readiness Checklist

- [ ] Snapshot collection tested on the exact PostgreSQL 18 minor release.
- [ ] Monitoring schema ownership and grants reviewed.
- [ ] pg_stat_statements enabled through approved restart/change process.
- [ ] I/O timing overhead tested and accepted.
- [ ] Snapshot retention and cleanup automated.
- [ ] Collector failure alert configured.
- [ ] Statistics reset detection included in dashboards/reports.
- [ ] OS storage metrics available for every PostgreSQL data/WAL/tablespace device.
- [ ] Cloud storage quota/IOPS/throughput metrics integrated where applicable.
- [ ] pg_wal free-space and replication-slot retention alerts configured.
- [ ] Temp-file growth alerts configured.
- [ ] Checkpoint and client-backend write/fsync trends baselined.
- [ ] Autovacuum backlog/freeze-risk monitoring configured.
- [ ] Incident bundle storage is access-controlled and has retention.
- [ ] Runbook tested in staging and during a controlled production exercise.

11.1 Go-Live Checklist

- [ ] Take a baseline during normal business traffic.
- [ ] Take a baseline during known batch/ETL/backup windows.

- [ ] Record expected top backend types and top SQL I/O producers.
- [ ] Verify alert routes and escalation contacts.
- [ ] Verify the operator can map PGDATA, pg_wal, temp tablespaces, and user tablespaces to OS/cloud volumes.
- [ ] Verify dashboard labels include cluster, host, database, device, and collection interval.

12. Operational Cheat Sheet

Question	Primary PostgreSQL Source	OS / Secondary Source
Who is doing I/O?	pg_stat_io, pg_stat_activity	pidstat -d
Which SQL?	pg_stat_statements	Application tracing/APM
Cache pressure?	pg_stat_io evictions, pg_statio_*	vmstat/free
Temp spill?	pg_stat_database, pg_stat_statements	temp filesystem df/iostat
Storage slow?	pg_stat_io timing	iostat/cloud volume metrics
Sequential scans?	pg_stat_user_tables	EXPLAIN (safe usage)
Checkpoint pressure?	pg_stat_checkpointer, pg_stat_io	iostat
Autovacuum?	pg_stat_progress_vacuum, pg_stat_io	iostat
WAL growth?	pg_stat_wal, slots, archiver, replication	df on pg_wal/cloud metrics
Long transactions?	pg_stat_activity	Application logs
Capacity/error?	catalog paths/tablespaces	df, findmnt, lsblk, kernel logs

12.1 Safe First Commands

```
SELECT version();
SELECT now(), * FROM pg_stat_io;
SELECT now(), * FROM pg_stat_checkpointer;
SELECT now(), * FROM pg_stat_bgwriter;
SELECT now(), * FROM pg_stat_wal;
SELECT now(), datname, blks_read, blks_hit, temp_files, temp_bytes FROM pg_stat_database;
SELECT pid, backend_type, state, wait_event_type, wait_event, query_start, xact_start,
    left(query,500) FROM pg_stat_activity WHERE pid <> pg_backend_pid();

# OS
findmnt -T "$PGDATA"
df -hT "$PGDATA"
```

```
iostat -xz 1 10
vmstat 1 10
```

13. Best Practices and Limitations

- Use deltas/rates, not raw lifetime counters, for incident attribution.
- Keep monitoring collection lightweight and predictable; one-minute collection is a practical starting point, not a mandatory interval.
- Do not reset statistics just to make reports easier. Preserve evidence during incidents.
- Do not treat average latency alone as sufficient; averages can hide tail latency and bursts.
- Do not treat sequential scans, autovacuum, checkpoints, or temp files as inherently bad. Determine whether they are expected and whether they cause service impact.
- Use query-level and object-level evidence before changing memory, checkpoint, vacuum, or index settings.
- Remember that pg_stat_io does not track every possible I/O path; for example, some relation I/O that bypasses shared buffers is not represented.
- PostgreSQL statistics and OS tools are complementary. Neither layer alone can prove the complete root cause on virtualized/cloud storage.
- Protect query text and incident bundles because they may contain sensitive values.

14. Official References

PostgreSQL 18 - Monitoring Database Activity: https://www.postgresql.org/docs/18/monitoring.html

PostgreSQL 18 - Cumulative Statistics System / pg_stat_io: https://www.postgresql.org/docs/18/monitoring-stats.html

PostgreSQL 18 - Run-time Statistics: https://www.postgresql.org/docs/18/runtime-config-statistics.html

PostgreSQL 18 - pg_stat_statements: https://www.postgresql.org/docs/18/pgstatstatements.html

PostgreSQL 18 - auto_explain: https://www.postgresql.org/docs/18/auto-explain.html

PostgreSQL 18 - Vacuuming Configuration: https://www.postgresql.org/docs/18/runtime-config-vacuum.html

PostgreSQL 18 - CHECKPOINT: https://www.postgresql.org/docs/18/sql-checkpoint.html

PostgreSQL 18 - Release Notes: https://www.postgresql.org/docs/18/release-18.html

Appendix A - Complete Snapshot/Delta SQL

This compact report summarizes the latest approximately one-minute interval by backend type/object/context. It refuses to hide a reset boundary: review stats_reset if values look inconsistent.

```
WITH latest AS (
  SELECT max(snapshot_ts) AS t2 FROM monitor.io_snapshots
), chosen AS (
  SELECT l.t2,
        (SELECT max(snapshot_ts)
         FROM monitor.io_snapshots s
         WHERE s.snapshot_ts <= l.t2 - interval '60 seconds') AS t1
  FROM latest l
), old AS (
```

```
 SELECT backend_type, object, context,
       sum(reads) reads, sum(coalesce(read_bytes, reads*op_bytes,0)) read_bytes,
       sum(read_time_ms) read_ms,
       sum(writes) writes, sum(coalesce(write_bytes, writes*op_bytes,0)) write_bytes,
       sum(write_time_ms) write_ms,
       sum(evictions) evictions, sum(fsyncs) fsyncs, sum(fsync_time_ms) fsync_ms,
       max(stats_reset) stats_reset
 FROM monitor.io_snapshots, chosen
 WHERE snapshot_ts = chosen.t1
 GROUP BY 1,2,3
), new AS (
 SELECT backend_type, object, context,
       sum(reads) reads, sum(coalesce(read_bytes, reads*op_bytes,0)) read_bytes,
       sum(read_time_ms) read_ms,
       sum(writes) writes, sum(coalesce(write_bytes, writes*op_bytes,0)) write_bytes,
       sum(write_time_ms) write_ms,
       sum(evictions) evictions, sum(fsyncs) fsyncs, sum(fsync_time_ms) fsync_ms,
       max(stats_reset) stats_reset
 FROM monitor.io_snapshots, chosen
 WHERE snapshot_ts = chosen.t2
 GROUP BY 1,2,3
)
SELECT n.backend_type, n.object, n.context,
      n.reads-o.reads AS reads,
      pg_size_pretty((n.read_bytes-o.read_bytes)::bigint) AS read_bytes,
      round((n.read_ms-o.read_ms)::numeric,2) AS read_ms,
      CASE WHEN n.reads>o.reads
         THEN round(((n.read_ms-o.read_ms)/(n.reads-o.reads))::numeric,3) END AS avg_read_ms,
      n.writes-o.writes AS writes,
      pg_size_pretty((n.write_bytes-o.write_bytes)::bigint) AS write_bytes,
      round((n.write_ms-o.write_ms)::numeric,2) AS write_ms,
      n.evictions-o.evictions AS evictions,
      n.fsyncs-o.fsyncs AS fsyncs,
      round((n.fsync_ms-o.fsync_ms)::numeric,2) AS fsync_ms,
      o.stats_reset AS old_reset, n.stats_reset AS new_reset
FROM new n JOIN old o USING (backend_type,object,context)
ORDER BY (n.read_bytes-o.read_bytes)+(n.write_bytes-o.write_bytes) DESC;
```

Appendix B - Incident Collection Shell Script

Use this as a starting point for a controlled incident bundle. Adapt connection options, output permissions, log redaction, and OS commands to local standards.

```
#!/usr/bin/env bash
set -euo pipefail
umask 077
```

```
TS=$(date +%Y%m%d_%H%M%S)
OUT=${1:-"/var/tmp/pg_io_$TS"}
mkdir -p "$OUT"

sql() { psql -X -v ON_ERROR_STOP=1 -P pager=off -c "$1"; }

sql "select version();" > "$OUT/00_version.txt"
sql "show track_io_timing; show track_wal_io_timing;" > "$OUT/01_settings.txt"
sql "select clock_timestamp(), * from pg_stat_io;" > "$OUT/10_pg_stat_io.txt"
sql "select clock_timestamp(), * from pg_stat_checkpointer;" > "$OUT/11_checkpointer.txt"
sql "select clock_timestamp(), * from pg_stat_bgwriter;" > "$OUT/12_bgwriter.txt"
sql "select clock_timestamp(), * from pg_stat_wal;" > "$OUT/13_wal.txt"
sql "select clock_timestamp(), * from pg_stat_database;" > "$OUT/14_database.txt"
sql "select
pid,usename,datname,backend_type,state,wait_event_type,wait_event,query_start,xact_start,left(query,1000)
query from pg_stat_activity where pid<>pg_backend_pid();" > "$OUT/20_activity.txt"
sql "select slot_name,slot_type,active,restart_lsn from pg_replication_slots;" > "$OUT/21_slots.txt" || true
sql "select * from pg_stat_archiver;" > "$OUT/22_archiver.txt" || true

command -v iostat >/dev/null && iostat -xz 1 10 > "$OUT/30_iostat.txt" || true
command -v vmstat >/dev/null && vmstat 1 10 > "$OUT/31_vmstat.txt" || true
command -v pidstat >/dev/null && pidstat -d 1 10 > "$OUT/32_pidstat.txt" || true
findmnt > "$OUT/40_findmnt.txt" 2>&1 || true
df -hT > "$OUT/41_df_hT.txt"
df -ih > "$OUT/42_df_ih.txt"
lsblk -f > "$OUT/43_lsblk.txt" 2>&1 || true

echo "Created $OUT"
echo Review_sensitive_SQL_text_before_sharing.
```